GOOGLE CLOUD HANDS-ON LAB

Infrastructure as Code with Terraform

Terraform: Create, Update, and Destroy Cloud Resources

Step-by-step practical documentation with screenshot placement guide

1. Overview

This lab uses Terraform to manage Google Cloud resources. You verify the Terraform installation, configure Google Cloud as the provider, create a Compute Engine virtual machine, update its network tags and machine type, and destroy the Terraform-managed infrastructure.

2. Objectives

- Verify Terraform installation.
- Define Google Cloud as the Terraform provider.
- Create Google Cloud resources with Terraform.
- Change VM tags and machine type.
- Destroy the resources created by Terraform.

3. Requirements and safety notes

- Use a browser and the assigned Google Cloud Skills lab.
- Use the temporary lab credentials, not your personal Google account.
- Use an Incognito/private browser window if possible.
- The lab is timed and cannot be paused after starting.
- Never include passwords or private credentials in screenshots shared publicly.

4. Task 1 — Sign in to Google Cloud Console

Step 1: Start the lab

Open the lab page and click Start Lab. Wait for the temporary credentials and Open Google Cloud console button.

Step 2: Open the Console

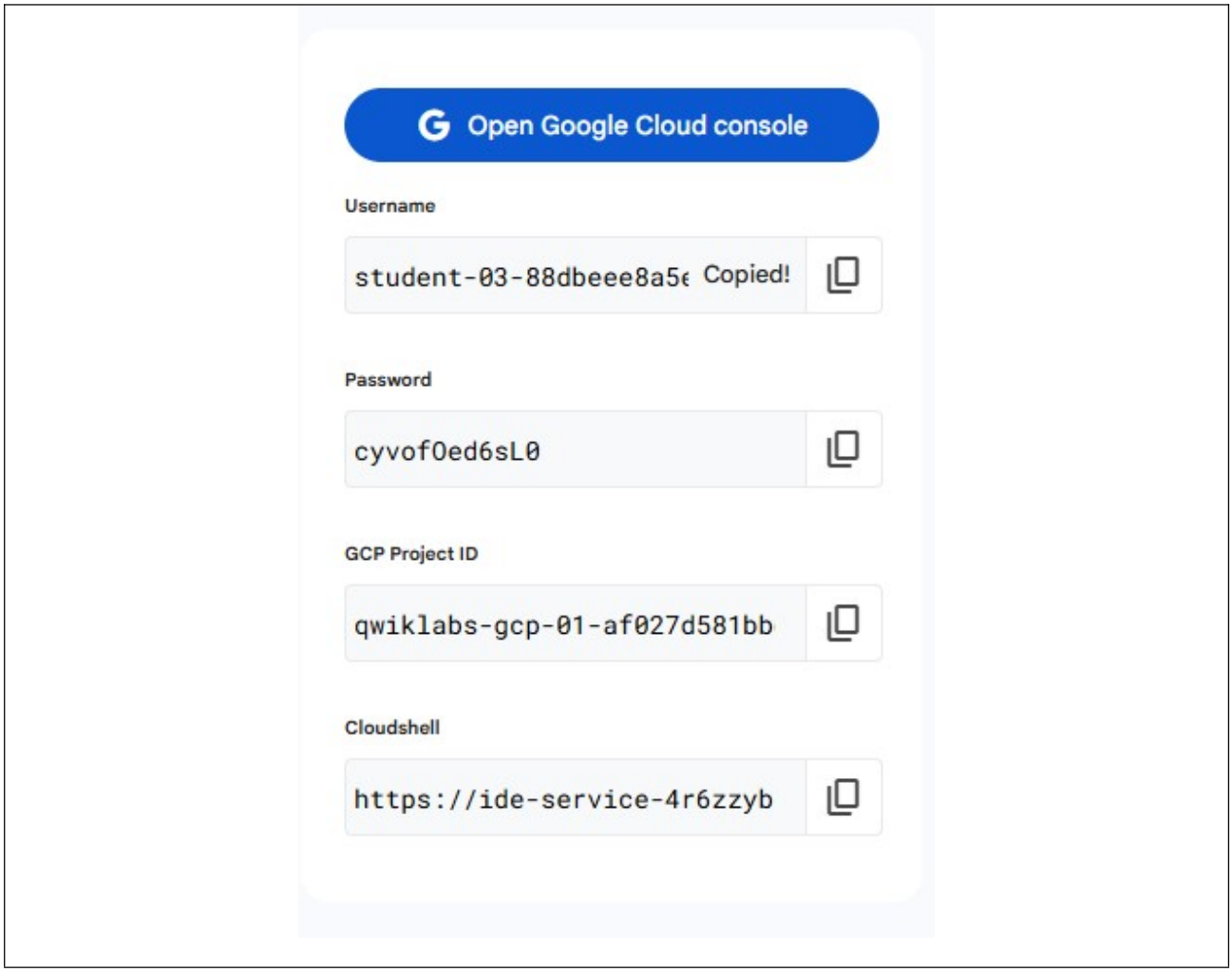
Click Open Google Cloud console. If prompted to choose an account, select Use another account.

Step 3: Sign in with temporary credentials

Copy the username and password from the lab access panel and enter them on the sign-in page. Use only the credentials provided by the lab.

Step 4: Complete initial prompts

Accept the terms if prompted. Do not add recovery options, enable two-factor authentication, or sign up for a free trial on this temporary account.



5. Task 2 — Set up Terraform and Cloud Shell

Step 5: Activate Cloud Shell

In the Google Cloud Console, click the Open Cloud Shell icon on the top toolbar, then click Continue. Wait for the terminal to connect.

Step 6: Verify the active account

```
gcloud auth list
```

The active account is marked with an asterisk.

Step 7: Verify the project ID

```
gcloud config list project
```

Confirm that the project shown is the project assigned to your current lab.

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to qwiklabs-gcp-04-4c4fa2b8f66c.
Use `gcloud config set project [PROJECT_ID]` to change to a different project.
student_03_88dbeee8a5e5@cloudshell:~ (qwiklabs-gcp-04-4c4fa2b8f66c)$ gcloud auth list
Credentialed Accounts

ACTIVE: *
ACCOUNT: student-03-88dbeee8a5e5@qwiklabs.net

To set the active account, run:
    $ gcloud config set account `ACCOUNT`

student_03_88dbeee8a5e5@cloudshell:~ (qwiklabs-gcp-04-4c4fa2b8f66c)$ gcloud config list project
[core]
project = qwiklabs-gcp-04-4c4fa2b8f66c

Your active configuration is: [cloudshell-2104]
```

Step 8: Install Terraform

The lab instructions state Terraform is not pre-installed in Cloud Shell. Run this block to configure the HashiCorp repository and install Terraform.

```
cat <<'EOF' > ~/.customize_environment
# Set up HashiCorp repository and install Terraform
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
EOF
bash ~/.customize_environment
```

Wait for installation to finish.

Step 9: Verify Terraform installation

```
terraform --version
```

The output should display the installed Terraform version.

```
student_03_88dbeee8a5e5@cloudshell:~ (qwiklabs-gcp-04-4c4fa2b8f66c)$ terraform --version
Terraform v1.16.6
on linux_amd64
```

6. Task 3 — Add the Google Cloud provider

Step 10: Create main.tf

```
touch main.tf
```

Click Open Editor in Cloud Shell. If needed, open the editor in a new window.

Step 11: Add provider configuration

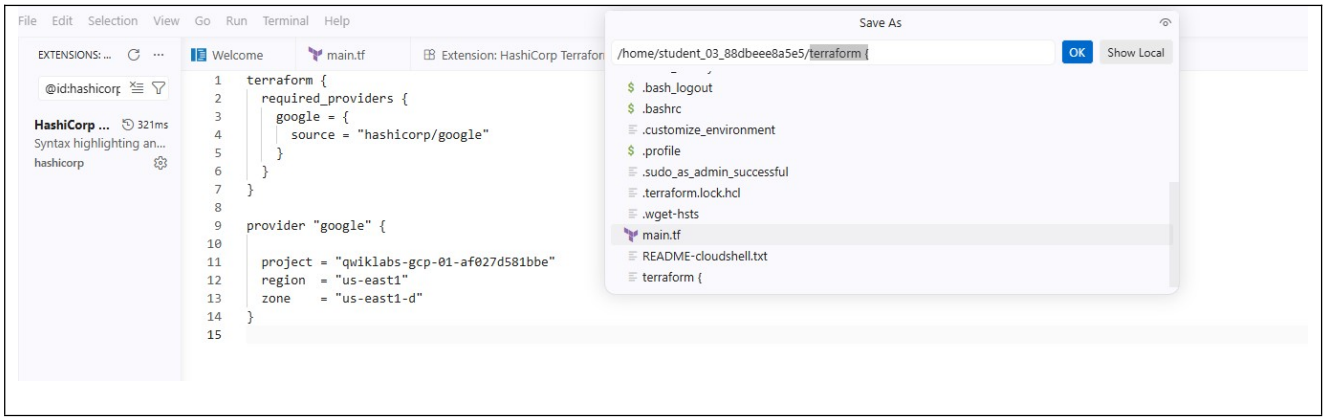
Paste this into main.tf. Replace Project ID, Region, and Zone with the values for your lab.

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
    }
  }
}

provider "google" {

  project = "qwiklabs-gcp-01-af027d581bbe"
  region  = "us-east1"
  zone    = "us-east1-d"
}
```

Select File > Save.



Step 12: Initialize Terraform

```
terraform init
```

Terraform downloads the Google provider and initializes the working directory. A successful run ends with a message indicating initialization succeeded.

```
student_03_88dbeee8a5e5@cloudshell:~ (qwiklabs-gcp-04-4c4fa2b8f66c)$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/google...
- Installing hashicorp/google v8.7.0...
- Installed hashicorp/google v8.7.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
```

7. Task 4 — Build the infrastructure

Step 13: Add the initial VM resource

In main.tf, add this resource. The first example intentionally omits the network interface for the lab exercise.

```
resource "google_compute_instance" "terraform" {
  name         = "terraform"
  machine_type = "e2-micro"
  boot_disk {
    initialize_params {
      image = "debian_image_project/debian_image_id_short"
    }
  }
}
```

Save the file. The image string is a placeholder as presented in the supplied lab text; use the exact valid image value specified by your lab if it causes an error.

Step 14: Run the first plan

```
terraform plan
```

The lab expects this initial configuration to fail because a Compute Engine instance requires at least one network_interface block.

```
student_03_88dbeee8a5e5@cloudshell:~ (qwiklabs-gcp-04-4c4fa2b8f66c)$ terraform plan

Error: Insufficient network_interface blocks

  on main.tf line 15, in resource "google_compute_instance" "terraform":
  15: resource "google_compute_instance" "terraform" {

At least 1 "network_interface" blocks are required.
```

Step 15: Add the network interface

Add this block inside the VM resource and save main.tf.

```
network_interface {
  network = "default"
```

```
  access_config {
  }
}
```

The final resource should include the VM name, machine type, boot disk, and network interface.

Step 16: Plan and apply the VM

```
terraform plan
```

Review the proposed changes. The plan should indicate that one VM will be created.

```
terraform apply
```

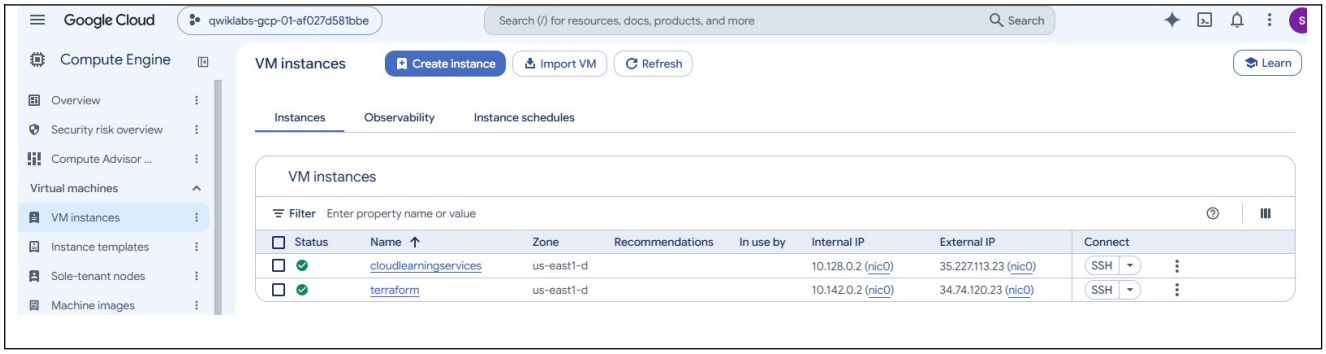
Review the plan and type yes to confirm. Wait until Terraform reports that the apply is complete.



Return to the lab page and click Check my progress for Build the infrastructure.

Step 17: Verify the VM in Console

In Google Cloud Console, navigate to Compute Engine > VM instances and find the instance named terraform.



8. Task 5 — Change the infrastructure

Step 18: Add network tags

Add the tags argument inside the google_compute_instance resource:

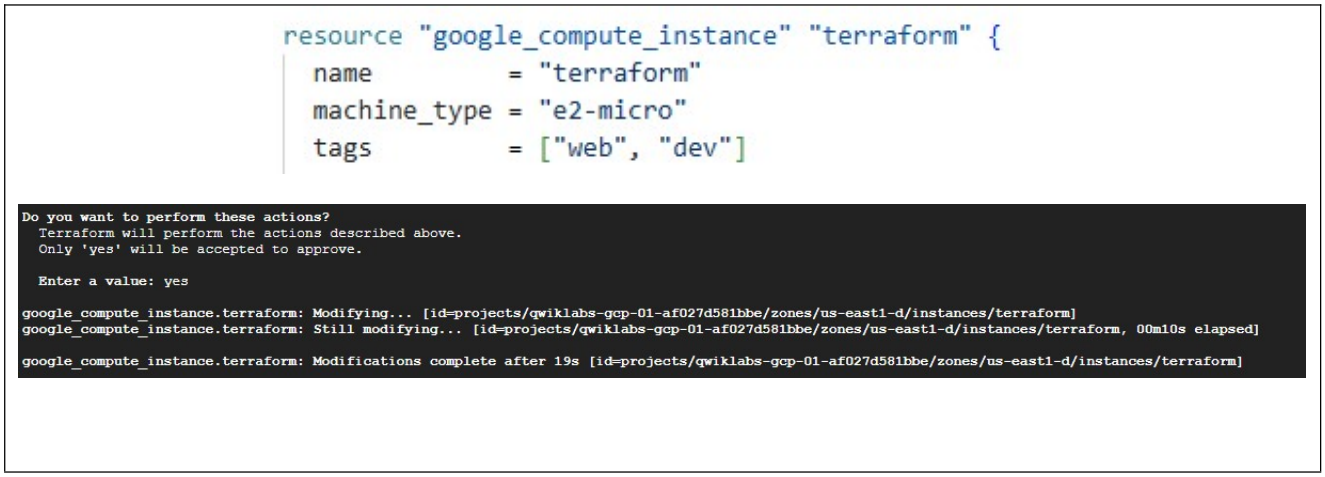
```
tags = ["web", "dev"]
```

Save main.tf, then run:

```
terraform plan

terraform apply
```

Review the changes and type yes when prompted. Terraform should add the web and dev tags to the VM.



Click Check my progress for Adding tags to the compute resource.

Step 19: Change the machine type

Change the machine type from e2-micro to e2-medium while keeping the tags.

```
machine_type = "e2-medium"
tags         = ["web", "dev"]
```

Save the file and run the following commands:

```
terraform plan

terraform apply
```

The supplied lab explains that changing the machine type of a running VM requires stopping it. The apply may fail unless you allow Terraform to stop the VM for this update.

Step 20: Allow stopping for update

Add this argument inside the google_compute_instance resource:

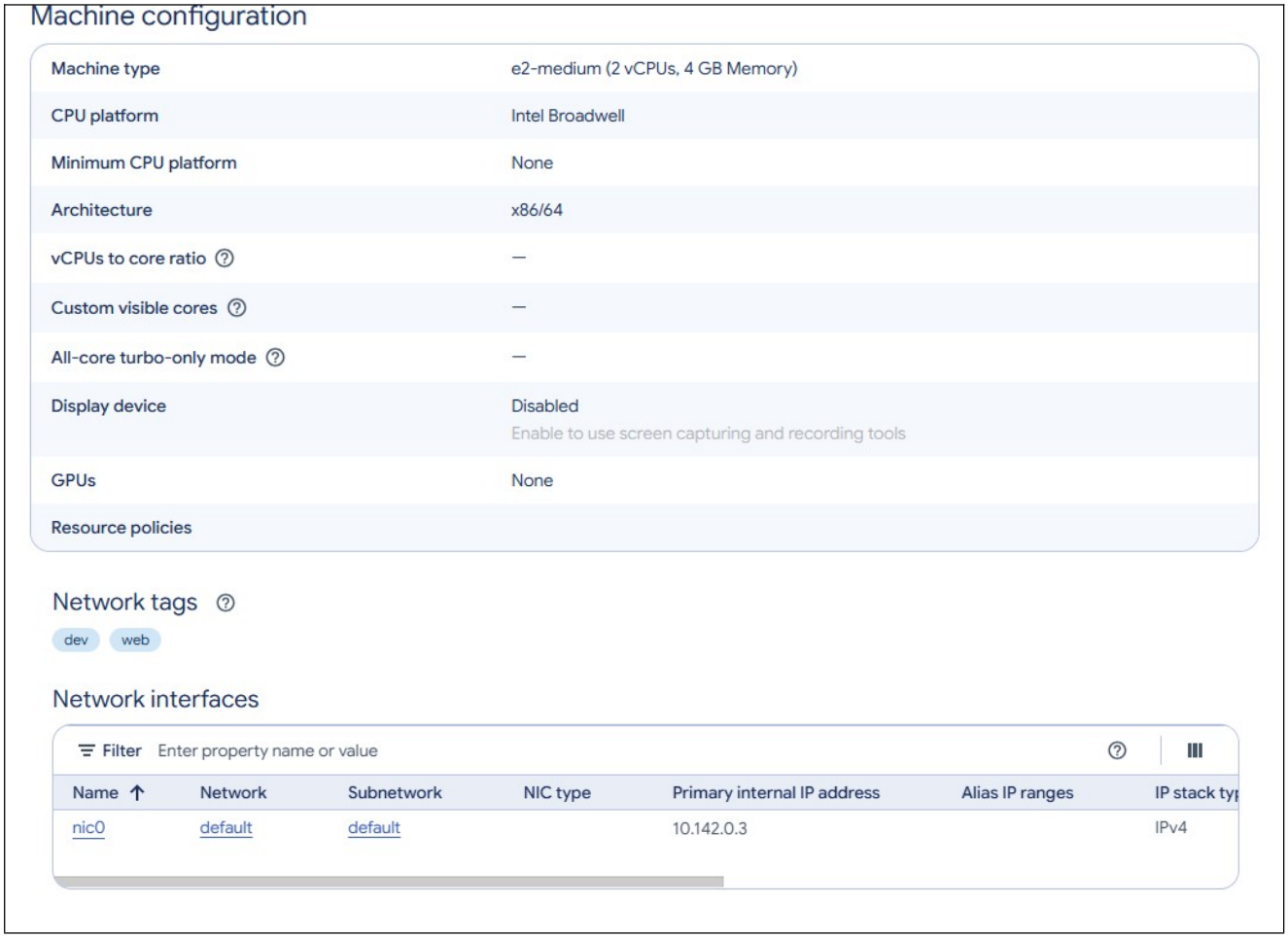
```
allow_stopping_for_update = true
```

Confirm that the resource still includes machine_type = "e2-medium", tags, boot disk, and network interface. Save the file.

```
terraform plan

terraform apply
```

Review the plan and type yes when prompted. Wait for Terraform to finish.



In Compute Engine > VM instances, open the terraform VM and verify the machine type and tags. Click Check my progress for the machine-type objective.

9. Task 6 — Destroy the infrastructure

Step 21: Destroy Terraform-managed resources

This deletes the infrastructure created during the lab. Confirm you are working in the temporary lab project and no longer need the VM or its data.

```
terraform destroy
```

Review the resources Terraform proposes to delete. Type yes only if the plan matches the lab resources you intend to remove.

Step 22: Verify deletion

Open Compute Engine > VM instances and verify the terraform VM no longer exists. Click Check my progress for Destroy the Infrastructure.

10. Conclusion

This lab demonstrates how Terraform defines and manages Google Cloud infrastructure. You configured the Google provider, initialized Terraform, created a Compute Engine VM, updated its tags and machine type, and destroyed the resources. Screenshots provide visual evidence of the main commands and results.

End of document